TecnoCampus

Universitat Pompeu Fabra Barcelona

Grau en Enginyeria Informàtica de Gestió i Sistemes d'Informació

Bones pràctiques en enginyeria de software: DDD, TDD, Clean/Hexagonal, CI/CD

Memoria Intermèdia

DANIEL BARBANCHO MONTERO

TUTOR: ALFREDO RUEDA UNSAIN

CURS ACADÈMIC 25-26

TecnoCampus Mataró-Maresme

Índex

Índex de figures 3

Índex de taules ..... Error! No s'ha definit el marcador.

1. Objecte del TFG 1

2. Estat de l'art: context, antecedents, necessitats d'informació 3

2.1 Context 3

2.2 Antecedents 4

2.2.1 Modular Architecture Hexagonal Demo Project from Alican Akkuş [6] 4

2.2.2 Example of Hexagonal Architecture and DDD from JonahathM2ndoza [4] 4

2.2.3 Hexagonal Architecture example from DomingoAlvarez99 [5] 5

2.3 Necessitats d'informació 5

2.3.1 Rehabilitació tradicional de la tendinopatia rotllana 5

2.3.3 Psiconeuroimmunologia en tendinopaties 6

2.3.4 Monitorització digital integrada 7

2.3.5 Open Wearables 7

2.3.6 Arquitectura Hexagonal 8

2.3.7 Ports/Adaptadors primaris i secundaris 9

2.3.8 Mapping 10

2.3.9 Avantatges de la Arquitectura Hexagonal 12

2.3.10 Desavantatges de la Arquitectura Hexagonal 12

2.3.11 DDD i l'Arquitectura Hexagonal 13

2.3.12 TDD (Test-Driven Development) 13

2.3.13 Control de versions 14

3. Objectius i abast 25

4. Metodologia 27

5. Definició de requeriments funcionals i tecnològics 29

5.1. Requeriments funcionals 29

5.2. Requeriments tecnològics 29

7. Desenvolupament 31

7.1 Inicialització del projecte 31

8. Bibliografia 49

Índex de figures

Figura 2.3.1. *Hexagonal architecture with business logic in the core ("application"), ports, adapters, and external components*. Font: [2].....9
Figura 2.3.2. *Hexagonal architecture: class diagram of primary port and adapter.* Font: [2].....9
Figura 2.3.3. *Hexagonal architecture: class diagram of secondary port and adapter.* Font: [2].....10
Figura 2.3.4. *Hexagonal architecture: duplication with two-way mapping.* Font: [2].....11
Figura 2.3.5. *Hexagonal architecture: duplication with one-way mapping.* Font: [2].....11
Figura 2.3.6. *Hexagonal architecture and DDD (domain-driven design).* Font: [2].....13
Figura 2.3.7. *Exemple de test per a TDD*. Adaptat de Font: [3].....14
Figura 2.3.8. *Exemple de codi mínim per a TDD*. Adaptat de Font: [3].....14
Figura 2.3.9. *Exemple de refactorització per a TDD*. Adaptat de Font: [3].....14
Figura 2.3.10. *Exemple d'una directiva en AngularJS*. Adaptat de Font: [23].....18
Figura 2.3.11. *Exemple d'un component en React*. Adaptat de Font: [20].....19
Figura 2.3.12. *A first test case. Adaptat de Font [25]*.....20
Figura
6.1.13. Diagrama Arquitectura Hexagonal del projecte. Font: Elaboració pròpia.....31
6.1.14. Diagrama mòdul de model. Font: Elaboració pròpia.....31
6.1.15. Diagrama de mòdul application. Font: Elaboració pròpia.....32
6.1.16. Diagrama de mòdul adapter. Font: Elaboració pròpia.....32
6.2.17. Diagrama d'entitat Patient. Font: Elaboració pròpia.....33
6.2.18. Diagrama d'entitat IndibaSession. Font: Elaboració pròpia.....34
6.2.19. Diagrama d'entitat PniReport. Font: Elaboració pròpia.....35
6.2.20 Diagrama d'entitat TrainingSession. Font: Elaboració pròpia.....36

6.3.21. Flux de treball per CreatePatient. Font: Elaboració pròpia.....37
6.3.22. Flux de treball de GetPatient. Font: Elaboració pròpia.....38
6.3.23. Flux de treball de GetIndibaFromPatient. Font: Elaboració pròpia.....39
6.3.24. Flux de treball ManagePolarConnection. Font: Elaboració pròpia.....41
6.3.25. Flux de treball PolarDailyScheduler. Font: Elaboració pròpia.....42
6.3.26. Flux de treball GetPatientTransitionRatio. Font: Elaboració pròpia.....44
6.3.27. Flux de treball GetWorkflowProgression. Font: Elaboració pròpia.....45

1. Objecte del TFG

La intenció d'aquest treball de fi de grau vol aportar a l'àmbit de la sanitat, oferint un programa per a facilitar l'emmagatzematge de registres de rehabilitacions de molt llarg termini de, per exemple, 24 mesos amb una tendinopatia crònica, que en aquest cas és en el que se centrarà aquest projecte.

Amb això, es pot millorar el monitoratge dels pacients amb aquesta mena de malalties per a aprofitar al màxim el procés de rehabilitació i que sigui moltíssim més convenient per a fer un traçament d'aquest.

Amb això, es pot millorar el monitoratge dels pacients amb aquesta mena de malalties per a aprofitar al màxim el procés de rehabilitació i que sigui moltíssim més convenient per a fer un traçament d'aquest.

La intenció d'aquest projecte és oferir un cas pràctic sobre l'aplicació d'aquestes diferents metodologies i arquitectures, així com oferir una estructura genèrica que es pot reutilitzar i ampliar en altres contextos professionals o acadèmics per a la facilitació de la creació de serveis escalables i fàcils de testejar.

A part d'ajudar a l'àmbit de la salut, també vol ajudar tant a l'àmbit acadèmic com a l'àmbit professional. Perquè seria un cas pràctic molt real que utilitzaria aquestes arquitectures i metodologies per a mostrar-les a futurs alumnes. A més de donar una estructura genèrica que pot ser reutilitzada en contextos més professionals per a la facilitació de la creació de serveis escalables.

2. Estat de l’art: context, antecedents, necessitats d’informació

2.1 Context

Aquest Treball de Fi de Grau (TFG) se centra en crear un sistema backend perquè els professionals de la fisioteràpia puguin registrar de manera digital i centralitzada les rehabilitacions de llarga durada, com la tendinopatia rotuliana crònica. El sistema inclou tant les sessions de radiofreqüència amb la màquina INDIBA com aspectes de psiconeuroimmunologia (PNI) i rehabilitació esportiva.

L’aplicació vol cobrir la necessitat de registrar dades d’aquestes tres àrees de la rehabilitació. Mitjançant una API de les màquines INDIBA i dels dispositius de PNI, la base de dades de l’aplicació es pot omplir amb aquesta informació.

Si no es pot accedir a les API o cal afegir informació sobre la rehabilitació esportiva, els fisioterapeutes poden introduir els detalls de cada sessió, com els paràmetres de tractament, notes o durada, a través d’un formulari al frontend. Aquestes dades s’enviaran al sistema backend per guardar-les.

El sistema també permet consultar l’historial complet d’un pacient, incloent les sessions, la PNI i la rehabilitació esportiva. Això facilita un seguiment detallat del procés i ajuda a prendre decisions clíniques basades en dades.

A més de la funcionalitat en salut, aquest TFG es basa en l’ús d’un conjunt d’arquitectures i bones pràctiques de desenvolupament:

- Arquitectura Hexagonal: Per aïllar la lògica de negoci dels detalls externs, assegurant millor escalabilitat i fàcil manteniment.
- DDD (Domain-Driven Design): Juntament amb l'anterior, per modelar l’aplicació al voltant del domini real de la fisioteràpia.
- TDD (Test-Driven Development): Per guiar el desenvolupament a través de proves abans de fer el codi, garantint la robustesa.
- Principis SOLID: Per garantir un disseny de codi net i flexible.
- CI/CD: Per establir un flux de treball automatitzat d'integració i desplegament.

2.2 Antecedents

2.2.1 Modular Architecture Hexagonal Demo Project from Alican Akkuş [6]

Aquest projecte ofereix una arquitectura hexagonal utilitzant dos microserveis.

Fa servir un total d'onze tecnologies diferents, algunes que serveixen per al testing i altres per a demostrar com es comuniquen en l'arquitectura.

El testing d'aquest projecte és complet, arribant a cobrir el cent per cent del codi. Tot això gràcies a la metodologia TDD. També utilitza diferents tipus de test, tant unitaris, integrats en Docker, de contracte tan extern com Consumer-driven i d'acceptació.

Quant a documentació explica que és l'arquitectura, motivació del projecte, tecnologies i una mica d'història sobre aquest. Però no explica en absolut com funciona el mateix.

En resum, és un bon projecte com a exemple d'ús de l'arquitectura hexagonal, encara que resulta una miqueta embullat navegar per aquest.

2.2.2 Example of Hexagonal Architecture and DDD from JonahathM2ndoza [4]

Aquest projecte utilitza, en canvi, tres microserveis per a demostrar l'arquitectura hexagonal.

Ho demostra usant dues tecnologies molt diferents de bases de dades: una relacional i una altra orientada a documents.

Quant a testing és molt pobre, de fet no té cap test, per la qual cosa no cobreix res del codi, a pesar que funcioni.

Quant a documentació sí que sembla contar l'ús de cadascun dels microserveis i dona una mica d'explicació sobre DDD i l'arquitectura hexagonal, però res més.

En aquest cas, el projecte sí que és una mica més fàcil de seguir, té noms fàcils i comprensibles per a saber on va cada funció a diferència de l'anterior que era una mica més complex.

2.2.3 Hexagonal Architecture example from DomingoAlvarez99 [5]

Aquest projecte, a diferència dels anteriors, demostra com funciona l'arquitectura hexagonal un sol microservei.

De manera semblant a l'anterior projecte, aquest demostra l'arquitectura hexagonal utilitzant dues tecnologies de bases de dades diferents: relacional i orientada a documents.

El testing d'aquest projecte és molt millor que el de l'anterior, fent servir tant tests unitaris com tests d'integració.

Quant a documentació, només explica com instal·lar el projecte, la seva configuració amb les bases de dades i com fer servir el testing.

A l'ésser el projecte més simple de tots, és bastant fàcil de poder entendre on va cada component del programa.

2.3 Necessitats d’informació

2.3.1 Rehabilitació tradicional de la tendinopatia rotllana

Els programes d’exercici supervisat constitueixen actualment el *gold standard* en el tractament de la tendinopatia rotuliana, incloent exercicis isomètrics, excèntrics i protocols de càrrega progressiva. L’evidència científica indica que els exercicis isomètrics poden reduir ràpidament el dolor (evidència de nivell A), mentre que els exercicis excèntrics i els programes de càrrega lenta i pesada mostren millores significatives de la funció a mitjà i llarg termini [37].

Tot i això, la manca d’adherència continua sent un problema recurrent: factors conductuals com la motivació baixa, creences negatives envers el dolor i limitacions de temps dificulten el seguiment del programa [38].

2.3.2 Tractament amb màquina INDIBA Activ

La radiofreqüència capacitiva–resistiva a 448 kHz (comercialitzada com INDIBA Activ) és una tècnica no invasiva i indolora que permet estimular en profunditat els teixits. Mitjançant l'aplicació d'un corrent altern de 448 kHz, es genera una hipertermia terapèutica que afavoreix la vasodilatació, l'augment del flux sanguini i l'activació de processos cel·lulars de reparació. Aquest conjunt d'efectes té com a objectiu accelerar la cicatrització i regeneració dels teixits lesionats [39].

Diferents estudis clínics donen suport al seu ús. Kumaran i Watson (2019), per exemple, van demostrar que vuit sessions de CRMRF (448 kHz) produïen millores significatives en el dolor i en la funció de pacients amb artrosi de genoll en comparació amb un grup placebo [40]. De manera similar, un altre assaig clínic va mostrar que la combinació d'INDIBA amb plasma ric en plaquetes reduïa el dolor associat a la condromalàcia rotuliana de manera més pronunciada que el placebo [41].

L'evidència científica suggereix que la radiofreqüència a 448 kHz pot modular positivament el dolor i potenciar la regeneració tisular en patologies cròniques del genoll [39]. Tot i això, la major part dels estudis disponibles es centra en els efectes immediats o a curt termini, i hi ha un buit important de recerca pel que fa a la seva aplicació sistemàtica dins protocols de rehabilitació prolongada de la tendinopatia rotuliana.

2.3.3 Psiconeuroimmunologia en tendinopaties

La psiconeuroimmunologia (PNI) estudia com l'estrès psicològic i els factors neuroendocrins influeixen tant en la resposta immune com en la percepció del dolor. L'estrès crònic activa l'eix hipotàlem–hipòfisi–adrenal (HPA), incrementant la secreció de cortisol, un mecanisme que pot alterar la regulació inflamatòria i potencialment retardar els processos de reparació del tendó. Paral·lelament, variables com la manca de son o l'ansietat incrementen la sensibilització al dolor i disminueixen la capacitat d'adaptació al tractament [42].

Tot i que la recerca específica sobre tendinopaties encara és limitada, els estudis en dolor crònic indiquen que la disfunció de l'eix HPA i de les vies neuroimmunes té un paper determinant en la perpetuació del dolor.

De manera coherent amb això, intervencions com el biofeedback, el control del ritme cardíac (biofeedback de variabilitat) o l'educació psicològica han mostrat millores en la reducció del dolor i en l'adherència al tractament en patologies musculoesquelètiques. En alguns casos, les teràpies cognitivoconductuals i les tècniques de relaxació s'han establert com a complements útils en pacients amb tendinopaties refractàries [42].

2.3.4 Monitorització digital integrada

L'avenç dels dispositius wearables (com monitors de ritme cardíac, acceleròmetres o sensors de moviment) juntament amb l'expansió de les aplicacions mòbils ha permès registrar dades de salut de manera contínua i contextual. En l'àmbit de la rehabilitació, aquests sensors s'utilitzen per mesurar el rang de moviment, la càrrega d'exercici i diversos paràmetres de biofeedback. Tot i així, la principal dificultat històrica ha estat la interoperabilitat i la gestió dels grans volums de dades generats per aquests dispositius.

Actualment, aquesta barrera es redueix gràcies a estàndards de dades sanitàries com HL7 FHIR (Fast Healthcare Interoperability Resources), que defineix models estandarditzats (com Patient, Observation o Device) accessibles mitjançant APIs RESTful [43].

La literatura recent identifica FHIR com la tendència principal per integrar dades procedents de dispositius en els sistemes de registre clínic. Per exemple, Ayaz et al. (2021) mostren que FHIR simplifica l'intercanvi d'informació entre aplicacions mòbils i sistemes d'història clínica electrònica (EHR) [43], mentre que Rendón-Moreno et al. (2025) destaquen la seva utilitat per incorporar dades de monitoratge domiciliari en fluxos clínics de forma estructurada [46].

2.3.5 Open Wearables

Open Wearables és una plataforma de codi obert pensada per unificar i facilitar l'accés a dades de salut personals de diferents dispositius wearable, com Garmin, Apple Health o Oura. El seu objectiu principal és solucionar la fragmentació de les integracions oferint una

infraestructura unificada que gestiona l'inici de sessió segur i fa que les dades tinguin el mateix format [45].

Aquesta eina ofereix als desenvolupadors una API unificada, ja sigui REST o GraphQL, que permet accedir a mètriques com la freqüència cardíaca, el son o l'activitat sense haver de crear connexions separades per a cada proveïdor. Això fa més fàcil treballar amb diferents formats de dades i límits d'ús dels fabricants [45].

La plataforma destaca per ser autoallotjable i funcionar sota una filosofia de "zero bloqueig de proveïdor", garantint que les dades de salut es mantinguin privades i sota el control del desenvolupador o l'usuari final. A més, incorpora funcions d'intel·ligència artificial per crear automatitzacions i recomanacions de salut utilitzant llenguatge natural, sense necessitat de processos complicats d'aprenentatge automàtic [45].

2.3.6 Arquitectura Hexagonal

L'Arquitectura de Ports i Adaptadors, o també anomenada Arquitectura Hexagonal, és una arquitectura creada per Alistair Cockburn l'any 2005 per tal que una aplicació pugui ser desenvolupada i testejada isolant les tecnologies que aquesta vol utilitzar (com bases de dades). [1]

Quan un esdeveniment l'hi arriba de l'exterior des d'un port, l'adaptador s'encarrega de transformar-lo en una trucada o missatge de procediment que passa a l'aplicació. I quan l'aplicació ha d'enviar un missatge, aquest passa per un port fins a un adaptador perquè l'exterior ho pugui entendre. [2]

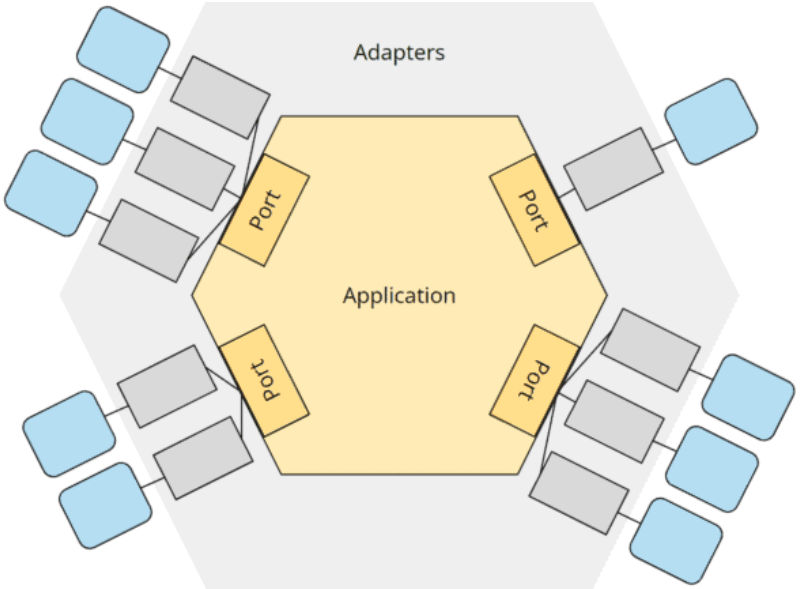


Figura 2.3.1. *Hexagonal architecture with business logic in the core ("application"), ports, adapters, and external components*. Font: [2]

2.3.7 Ports/Adaptadors primaris i secundaris

Aquests ports i adaptadors poden ser categoritzats com a driving (o primaris), que normalment són aquells d'entrada, i driven (secundaris) que són els de sortida [2].

Els ports i adaptadors primaris son aquells que "controlan" l'aplicació. Els adaptadors utilitzen els ports primaris, i aquests son implementats a un servei.

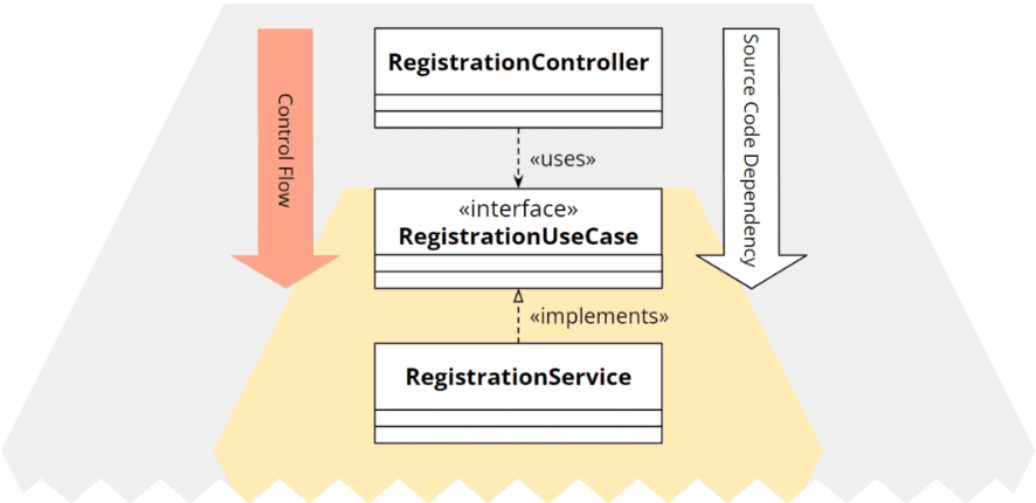


Figura 2.3.2. *Hexagonal architecture: class diagram of primary port and adapter.* Font: [2]

En el cas dels ports i adaptadors secondaris, son aquells que són “controlats” per l’aplicació. Els serveis utilitzan els ports secundaris que a la vegada són implementats al adaptador secundari.

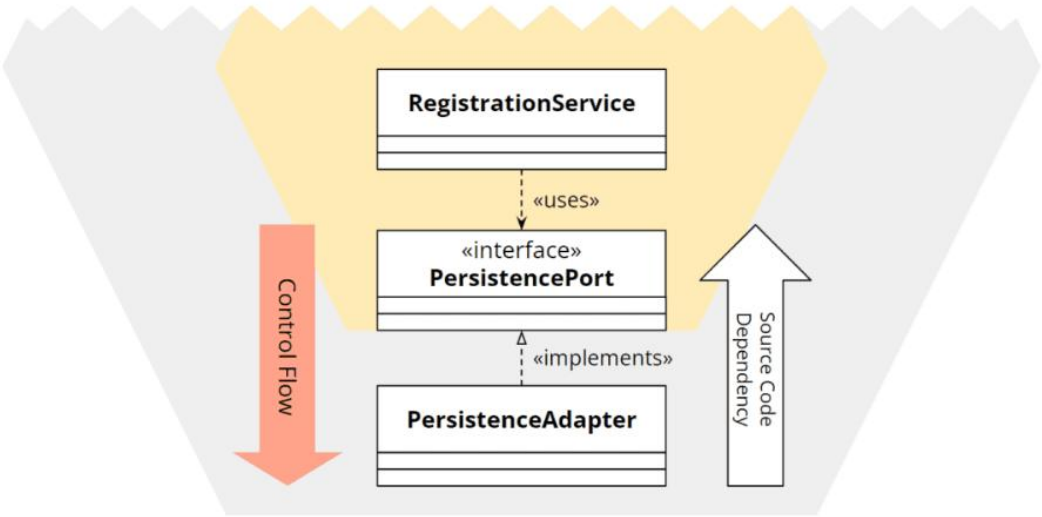


Figura 2.3.3. *Hexagonal architecture: class diagram of secondary port and adapter.* Font: [2]

2.3.8 Mapping

El problema és que, encara que es busca mantenir separada la part important del programa, la lògica del negoci, dels detalls tècnics, en la pràctica moltes vegades acaben barrejant-se. Això provoca que aquestes classes deixin de ser només la lògica del programa i passin a dependre de la base de dades. Per solucionar aquest problema s’utilitzen dos formes: duplicació amb mapping de doble sentit o duplicació amb mapping de un únic sentit [2].

La duplicació amb mapping de doble sentit crea una entitat exclusiva que no conté cap lògica de negoci, però que, en canvi, sí que conté tots els detalls tècnics. L’adaptador doncs s’encarrega de mapejar les dues entitats.

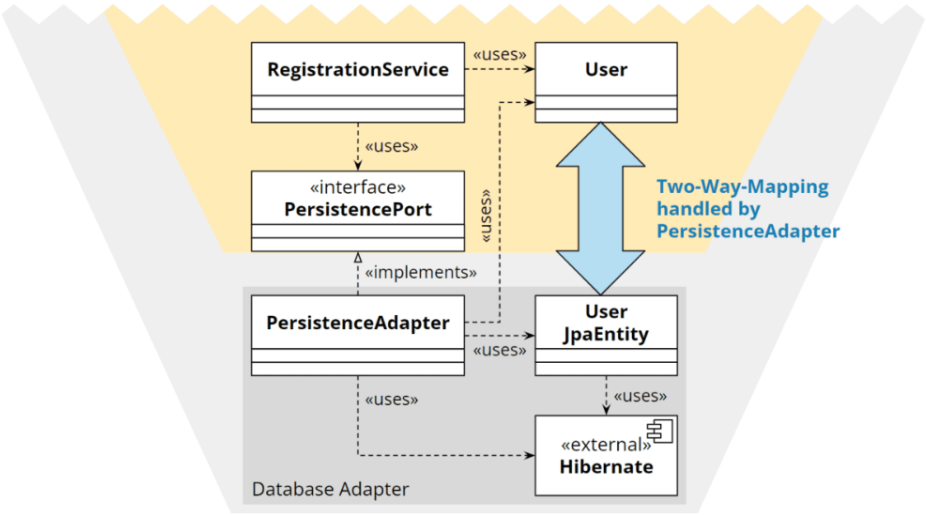


Figura 2.3.4. *Hexagonal architecture: duplication with two-way mapping.* Font: [2]

La duplicació amb mapping d'un únic sentit, en canvi, fa que l'entitat de persistència sigui una interfície que tant l'entitat de l'adaptador com la del nucli, la utilitzin. D'aquesta manera és possible només enviar la classe directament.

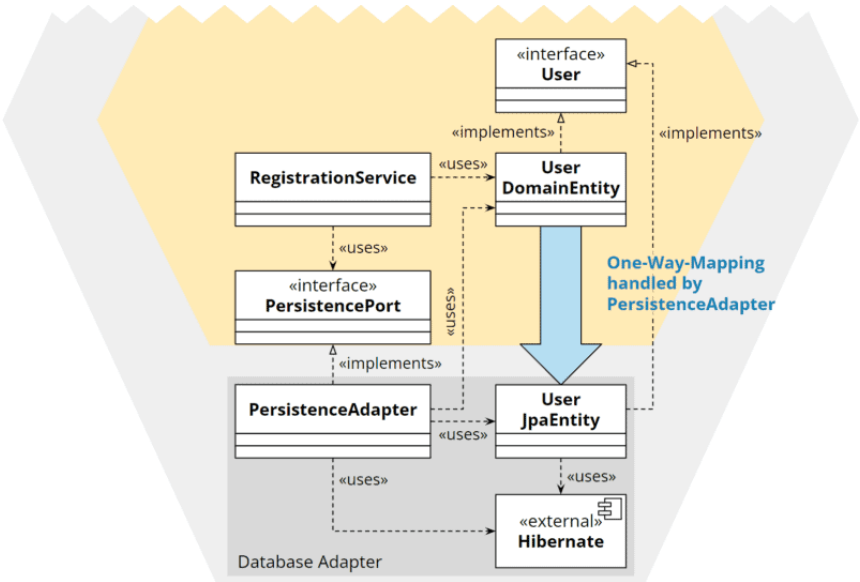


Figura 2.3.5. *Hexagonal architecture: duplication with one-way mapping.* Font: [2]

2.3.9 Avantatges de la Arquitectura Hexagonal

L'arquitectura Hexagonal ofereix diversos avantatges en el desenvolupament d'un projecte tecnològic: [2]

Flexibilitat: Es pot substituir la base de dades, la interfície o qualsevol tecnologia externa sense necessitat de modificar la lògica principal del programa. D'aquesta manera, el sistema no depèn d'una tecnologia específica i resulta més senzill d'actualitzar en el futur.

Aïllament: La lògica del programa es manté separada dels detalls tècnics. El nucli se centra exclusivament en les decisions de negoci, sense dependre de bases de dades, frameworks o interfícies. Aquesta separació evita la confusió i contribueix a mantenir el codi més net.

Desenvolupament: Cada component del sistema es pot desenvolupar de manera independent. Diferents membres de l'equip poden encarregar-se respectivament de la base de dades, la interfície i la lògica interna sense interferències. Aquesta divisió accelera el desenvolupament i redueix la probabilitat d'errors.

Facilitat de testeig: La lògica del sistema es pot provar sense necessitat d'una base de dades, un servidor o una interfície real. Aquesta característica facilita la detecció primerenca d'errors i simplifica el procés de prova.

2.3.10 Desavantatges de la Arquitectura Hexagonal

La implementació de l'arquitectura hexagonal implica un esforç addicional en comparació amb el desenvolupament d'una aplicació senzilla, ja que requereix la creació de ports, adaptadors i mapatges entre classes. Aquesta inversió sol ser beneficiosa en projectes de gran envergadura, mentre que en aplicacions petites i simples pot no resultar justificada [2].

Es recomana disposar d'una persona amb experiència que pugui avaluar si aquesta arquitectura és adequada per al projecte concret.

Treballar amb un projecte d'exemple ajuda a veure com funciona a la pràctica i permet guanyar experiència per decidir si convé aplicar-la en futurs desenvolupaments.

2.3.11 DDD i l’Arquitectura Hexagonal

L’arquitectura hexagonal es representa habitualment amb un hexàgon central que pot incloure “entitats”, “casos d’ús” o un “model de domini”. No obstant això, aquesta arquitectura no determina explícitament què s’ha d’incloure en el seu interior. Segons Alistair Cockburn, l’aspecte fonamental consisteix a envoltar l’aplicació amb una API i realitzar les proves des de l’exterior. El domain-driven design (DDD) resulta especialment adequat en aquest context, ja que facilita l’organització de les regles de negoci dins l’hexàgon. En els propers articles, utilitzaré aquest hexàgon del domini per exposar com es pot implementar l’arquitectura hexagonal amb Java [2].

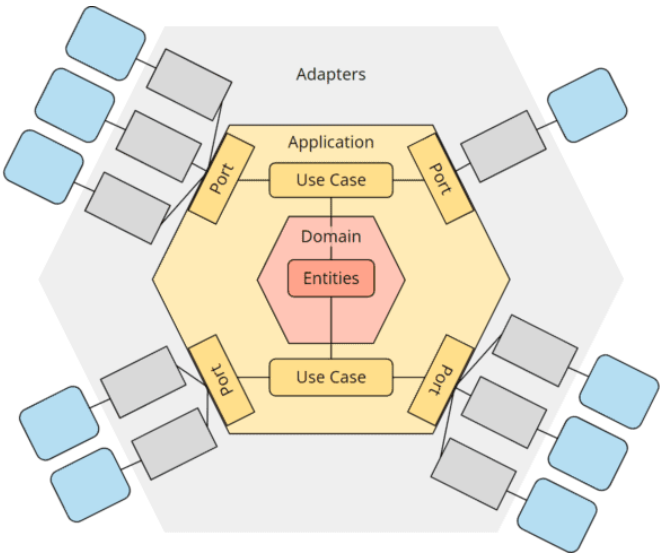


Figura 2.3.6. *Hexagonal architecture and DDD (domain-driven design).* Font: [2]

2.3.12 TDD (Test-Driven Development)

El Test-Driven Development es una metodología de desenvolupament d’aplicacions on, en lloc d'escriure el codi i després provar-lo, el procés exigeix escriure primer la prova automàtica i en acabat el codi necessari per a satisfer-la [3]. Té tres fases:

Fer un test que falli (Red): Escriure un test automàtic per a una funcionalitat que encara no existeix. En executar-ho, ha de fallar (o ni tan sols compilar). Això obliga a definir la interfície i el comportament esperat des del punt de vista de l'usuari de la classe [3].

```
public void testSuma() {
   assertEquals(5, Matematicas.suma(2,3));
}
```

Figura 2.3.7. *Exemple de test per a TDD*. Adaptat de Font: [3]

Fer el codi mínim (Green): Escriure exclusivament el codi imprescindible perquè el test passi. No importa si la solució és "lletja" o temporal [3].

```
public class Matematicas {
   public static int suma (int a, int b) {
      return 5;
   }
}
```

Figura 2.3.8. *Exemple de codi mínim per a TDD*. Adaptat de Font: [3]

Arreglar el codi (Refactor): Una vegada el test passa, es neteja el codi. S'eliminen duplicitats i es millora la implementació assegurant que els tests continuen passant [3].

```
public class Matematicas {
   public static int suma (int a, int b) {
      return a+b;
   }
}
```

Figura 2.3.9. *Exemple de refacorització per a TDD*. Adaptat de Font: [3]

2.3.13 Control de versions

Un control de versions és aquells sistemes que ajuden a realitzar un seguiment dels canvis realitzats al llarg del desenvolupament d'un projecte, que permet recuperar versions anteriors en cas que hi hagi conflicte amb noves modificacions [7].

2.3.14 GitHub

GitHub és una plataforma web per al control de versions que utilitza Git com el seu sistema principal. GitHub ofereix un disseny agradable per a poder veure el codi del repositori i les seves versions anteriors, però a més ofereix eines molt útils com GitHub Copilot per a suggeriments de codi i Pull Requests per a tenir notificacions de canvis entre repositoris i GitHub Actions per a automatitzar "workflows" en els repositoris. A més, es pot utilitzar amb tecnologies incorporades com Github Desktop que deixa fer "pulls" i "pushes" amb un senzill clic del ratolí.

També té integrat un sistema de projectes per a poder veure el seu progrés amb diversos estils, incloent-hi un model Kanban. Igualment, es pot fer servir el sistema de "Issues" per a poder fer un seguiment de què cal fer en el projecte.

2.3.15 GitLab

Semblant a GitHub, GitLab és una plataforma web dedicada al control de versions que utilitza Git com el seu sistema. GitLab ofereix una plataforma de DevOps que també implementa "pipelines" per a assegurar el CI/CD del projecte i assegurar la seguretat de l'aplicació amb reports de vulnerabilitat.

A més, deixa crear projectes des de zero en la seva pàgina web, incloent-hi plantilles per a diferents tipus de tecnologies, entre elles Java Spring Boot. També es pot afegir "Wikis" per a documentar el funcionament de l'aplicació.

2.3.16 Java Spring Boot

Per a facilitar la creació i el desplegament de l’aplicació, s’utiltiza Java Spring Boot. Elimina molt del treball en configuració manual de dependències i automàticament es configura amb llibreries de Spring i de tercers.

Gràcies a això, es pot desplegar de manera ràpida i còmoda l’aplicació per a poder presenciar el seu complet funcionament [9].

2.3.17 Maven

Spring Boot per a poder iniciar ràpidament un projecte de Java utilitza Maven. Aquesta eina és la que permet poder començar una aplicació a Java en qüestió de segons, per exemple, gestiona les dependències de manera automàtica. A més, disposa d'informació sobre la qualitat del projecte, reports de tests unitaris o dependències usades en el projecte [10].

2.3.18 Spring Data

Per a poder accedir a les dades que ens anirà proveint l'usuari, s'utilitza Spring Data. Aquesta eina que proporciona Spring facilita l'accés a dades amb bases de dades tan relacionals com amb no relacionals [11].

2.3.19 Spring Data JPA

Spring Data JPA ajuda a implementar repositoris per a l'accés de dades, reduint introduir codi repetitiu en consultes senzilles i també redueix l'esforç per a implementar la capa d'accés de dades. Bàsicament fent que Spring s'encarregui de connectar tot automàticament [12].

S'utilitza aquest mòdul principalment per a bases de dades relacionals.

2.3.20 Spring Data MongoDB

Spring Data MongoDB és l'eina que proporciona la integració amb una base de dades MongoDB, oferint un model de programació consistent i familiar amb tota la resta que estigui basat en Spring [13].

S'utilitza aquest mòdul per a poder comunicar amb una base de dades no relacional.

2.3.21 H2 Database Engine

Spring Boot també inclou un sistema gestor de base de dades relacional incorporada per a provar l'aplicació en local. Les dades es guarden en memòria en comptes d'en el disc, per la qual cosa no es mantindran una vegada s'apagui l'aplicació. És un gestor molt ràpid, de codi obert, escrit a Java, utilitza SQL estàndard i està encriptat [17].

Això facilita provar la lògica de l'aplicació sense haver d'obrir una base de dades independent, agilitzant el procés de prova.

2.3.22 Swagger

Swagger és bàsicament un framework dedicat a documentar i provar APIs RESTful de manera estandarditzada. Podent donar feedback, utilitzar anomenades REST i documentació des de la seva pàgina web [14].

A diferència de Postman, no fa falta instal·lar cap programa addicional per a poder provar crides.

2.3.23 HTML 5

El llenguatge de marcat d'hipertext (HTML) s'utilitza per a crear el que és els fonaments d'una pàgina web. Es dedica a estructurar on quedaran cada element dins d'aquesta.

La versió més actual, que permet interactuar amb dades locals i amb els servidors de manera més senzilla, i a part, proporciona major suport multimèdia. Aquesta versió, a diferència de l'anterior, no depèn de APIs ni complements a tercer, cosa que permet que carregui més ràpidament i assegurar la compatibilitat amb tots els navegadors [18].

2.3.24 CSS 3

Els Fulls d'estil en Cascada (CSS) és un llenguatge de disseny gràfic per a les aparences de pàgines web escrites en HTML o de documents XML.

L'última versió CSS3 aporta més control sobre els elements que les anteriors versions, afegint noves propietats a vores, fons, colors, text i interfícies. També afegeix selectors per atributs, degradats lineals i radials amb repetició i sense, models de caixes bàsics i altres propietats com afegir animacions i media queries [19].

2.3.25 AngularJS

AngularJS és un framework de JavaScript desenvolupat per Google, dissenyat específicament per a construir aplicacions web amb una arquitectura robusta i altament mantenible utilitzant objectes JavaScript estàndard (POJO) [23]

Utilitza una arquitectura Model-Vista-Controlador (MVC), o, en el cas específic de AngularJS, Model-Vista-ViewModel (MVVM), que ajuda a separar la lògica de la vista i facilita el manteniment i l'ampliació de les aplicacions. Un avantatge important és que utilitza objectes JavaScript simples en el model, això evita recórrer a mètodes complexos i permet unir dades de diferents llocs de manera senzilla [23].

La filosofia d'aquest framework es basa en l'ús de codi declaratiu per a la construcció d'interfícies d'usuari, estenent el vocabulari HTML mitjançant directives per a fer el codi més expressiu i llegible [23].

```
 myModule.directive('myComponent',
function(mySharedService) {
    return {
        restrict: 'E',
        controller: function($scope, $attrs, mySharedService)
{
            $scope.$on('handleBroadcast', function() {
                $scope.message = 'Directive: ' +
mySharedService.message;
            });
        },
        replace: true,
        template: '<input>'
    };
 });
```

Figura 2.3.10. *Exemple d'una directiva en AngularJS*. Adaptat de Font: [23]

AngularJS gestiona automàticament la sincronització entre el model i la vista a través de l'enllaç de dades bidireccional (two-way data binding) i utilitza la injecció de dependències per a facilitar el desenvolupament i les proves (testing) [22].

2.3.26 React

React és un framework de JavaScript creat per enginyers en Facebook per a desenvolupar interfícies d'usuari complexes amb conjunts de dades que s'anaven canviant contínuament [21].

React està orientat a la construcció d'interfícies d'usuari (UI) tant per a plataformes web com natives. La seva filosofia de disseny es fonamenta en l'ús de components, que són unitats de codi aïllades, que es poden reutilitzar per compondre les diferents interfícies [20].

```
function VideoList({ videos, emptyHeading }) {
  const count = videos.length;
  let heading = emptyHeading;
  if (count > 0) {
    const noun = count > 1 ? 'Videos' : 'Video';
    heading = count + ' ' + noun;
  }
  return (
    <section>
      <h2>{heading}</h2>
      {videos.map(video =>
        <Video key={video.id} video={video} />
      )}
    </section>
  );
}
```

Figura 2.3.11. *Exemple d’un component en React*. Adaptat de Font: [20]

Aquesta tecnologia utilitza una sintaxi que integra la lògica de programació amb el marcatge (JSX). Aquesta integració permet que React gestioni la renderització de manera eficient i actualitzi la pantalla únicament quan es produeixen canvis en les dades [20].

2.3.27 JUnit 6

JUnit 6 és la versió més actual del framework per el testatge d'aplicacions que utilitzin Java. És la combinació dels mòduls JUnit Platform, que és la fundació per a iniciar el framework en l'aplicació, JUnit Jupiter que és el model d'extensió i programació per a escriure els tests i JUnit Vintage que ofereix un motor de testatge per a poder córrer tests fets en anteriors versions (JUnit 3 i 4) [24].

Per a poder utilitzar es requereix fer servir l'anotació @test, la qual cosa indica que el codi usarà JUnit per a provar l'aplicació [25].

```
import static org.junit.jupiter.api.Assertions.assertEquals;

import example.util.Calculator;

import org.junit.jupiter.api.Test;

class MyFirstJUnitJupiterTests {

    private final Calculator calculator = new Calculator();

    @Test
    void addition() {
        assertEquals(2, calculator.add(1, 1));
    }

}
```

Figura 2.3.12. *A first test case. Adaptat de Font [25]*

2.3.28 Testcontainers

Testcontainer és una llibreria de codi obert que ofereix contenidors de Docker que es poden crear i eliminar quan no siguin necessàries. Això fa que no faci falta mocks ni configuracions complicades d'entorn per a la fase de testing [26].

Per a poder utilitzar-la, cal afegir la dependència, en el cas que s'usi en una aplicació Maven. En el test es pot llavors definir el contenidor Docker que es necessiti i algunes línies de codi per a poder activar i desactivar una vegada acabin els tests [27].

2.3.29 GitHub Actions

GitHub Actions és una plataforma de CI/CD integrada ja en GitHub. Permet fer "workflows" per a construir, testar i desplegar codi en un repositori GitHub que s'activen per un esdeveniment (com per exemple un "push" d'una branca) [28].

GitHub Actions té diversos components [29]:

Workflows: Un procés automatitzat configurable que executarà treballs. Es defineixen en un arxiu YAML dins del repositori i s'executa quan ocorre un esdeveniment.

Esdeveniment: Es defineix un esdeveniment com una activitat que ocorre dins d'un repositori, per exemple, un push en una branca.

Treball: Són els passos dins d'un Workflow, que pot ser un script shell o una acció que s'executarà dins. Van en ordre i són dependents l'un de l'altre.

Acció: És un conjunt predeterminat i reutilitzable que fa una tasca específica dins del Workflow, un exemple pot ser extreure un repositori Git de GitHub.

Executors: És un servidor on s'executen els Workflows, podria ser, per exemple, un contenidor en Docker amb un Ubuntu Linux.

2.3.30 GitLab Auto DevOps

GitLab Auto DevOps és un conjunt d'eines integrades en GitLab per a automatitzar el procés de CI/CD sense configuració manual. Encara que aquest pot funcionar en repositoris de GitHub [30].

Per a activar un Auto DevOps s'ha d'anar al teu perfil de GitLab, en la part de l'esquerra, i trobar el teu projecte.

Seleccionar Propietats -> CI/CD

Expandir l'opció d'Auto DevOps

Activar la casella de Predeterminat per a pipeline d'Acte DevOps.

Opcionalment, pots afegir un domini basi i una estratègia de llançament.

Finalment guardar els canvis.

2.3.31 IntelliJ IDEA

IntelliJ IDEA és el IDE líder per al desenvolupament professional a Java i Kotlin [31]. Algunes de les coses que ofereix aquest entorn són: Ajudar a escriure codi repetitiu gràcies a la codificació ràpida, injeccions de llenguatge, descompiladors integrats, cerca a tot arreu del projecte, compatibilitat amb GitHub i amb bases de dades, execució d'aplicacions amb un sol clic i finalització de codi intel·ligent [32].

2.3.32 Visual Studio Code

Visual Studio Code és un entorn de desenvolupament de codi obert que suporta la gran majoria de llenguatges de programació, tant de backend com de frontend. Ofereix una biblioteca d'extensions per a ajudar en el desenvolupament i és molt personalitzable [33].

2.3.33 Trello

Trello és una eina d'organització basada en taulers i targetes a estil Kanban. Aquest té integracions amb diverses aplicacions externes, però la més interessant en aquest cas és la seva integració amb GitHub, on pots ajuntar Pull Requests, Issues i commits a una targeta [34].

2.3.34 GitHub Projects

GitHub Projects és l'eina de productivitat integrada en els repositoris de GitHub. Aquesta integra directament les Issues i Pull Requests dels repositoris directament, també té diverses plantilles de planificació, incloent-hi una en estil Kanban [35]

2.3.35 Polar AccessLink API

Polar AccessLink facilita la recuperació de dades d'entrenament i activitat diària dels dispositius Polar.

AccessLink utilitza el protocol OAuth 2.0 per a l'autenticació. Els socis registrats han de fer servir les credencials d'inici de sessió designades per sol·licitar autorització i accedir a les dades de l'usuari.
Tant els formats de fitxer JSON com els formats XML són compatibles; no obstant això, només es permet el format JSON durant l'autenticació OAuth2.

Si teniu un compte de Polar Flow, podeu establir un client API per a AccessLink enviant el formulari de sol·licitud disponible a https://admin.polaraccesslink.com.

És necessari concedir tots els permisos necessaris per utilitzar les seves dades. Aquests permisos es poden modificar en qualsevol moment a través de https://account.polar.com. [46]

Per començar, has de seguir aquests passos [46]:

- Crear un compte de Polar Flow, en cas que no en tinguis cap.
- Anar a https://admin.polaraccesslink.com i loguejar-te amb les teves credencials
- Omplir la informació del teu servei o aplicació i crear el client.
- Una vegada el client ha estat creat, recorda de copiar la id i el secret del client OAuth2.
- Obtenir el codi d’autorització per l’usuari
- Utilitzar el codi d’autorització per obtenir el token d’accés per l’usuari
- Registrar l’usuari al nostre client per poder accedir a les seves dades.

3. Objectius i abast

3.1 Objectius del client

- Oferir un monitoratge complert d'un pacient amb una rehabilitació de llarga duració de tendinopatía crònica.
- Digitalitzar el sistema d'emmagatzematge de les sessions INDIBA per a ajudar al traçament de pacients i futures anàlisis de dades per a publicacions científiques d'alt impacte.
- Obtenir un projecte de qualitat per a donar exemple com a cas pràctic real de la utilització d'arquitectura hexagonal i bones pràctiques de programació.
- Presentar un projecte que serveix com a estructura genèrica d'arquitectura hexagonal per a altres propòsits de context tant professional com acadèmic.

3.2 Objectius del producte o servei

- Desenvolupar una aplicació web capaç de registrar: una rehabilitació completa, associades a cada pacient, utilitzant arquitectura hexagonal, DDD, TDD, CI/CD i respectant els principis SOLID.
- Desenvolupar un sistema que es nutri d'APIs externes per proporcionar les dades necessàries per el monitoratge de la rehabilitació.
- Proporcionar un formulari intuïtiu per a la introducció de dades clíniques d'una sessió INDIBA per part del fisioterapeuta. Emmagatzemant les dades en una base de dades relacional o no relacional, depenen de la tecnología d'emmagatzematge utilitzada.
- Permetre la consulta de l'historial de sessions de cada pacient de manera estructurada.

3.3 Usuari final

- Fisioterapeutes que utilitzen l'eina per registrar i consultar l'evolució clínica dels seus pacients sense perdre temps en processos complexos.
- Professionals sanitaris que estiguin interessats en analitzar les dades per fer publicacions científiques
- Alumnes universitaris que estiguin aprenent arquitectura hexagonal i bones pràctiques.

- Desenvolupador professionals que volen tenir una estructura per començar els seus projectes amb arquitectura hexagonal.

4. Metodologia

4.1 Planificació

Per a planificar el treball que es fa en aquest projecte, s'utilitza una metodologia Agile amb un tauler Kanban, és a dir, el treball s'organitza en etapes setmanals anomenades "sprint", fent servir l'eina en línia Trello.

En un sprint sempre es marca aquest cicle que es repeteix fins al final del projecte:

Inici de sprint: En aquesta fase es fa una repassada a les tasques que quedin per fer dins del backlog, aquí se seleccionen aquelles que es fan durant aquest sprint.

Desenvolupament del sprint: Aquesta fase és on es comença a treballar en les tasques seleccionades de l'inici del sprint. A mesura que es vagin completant tasques, s'actualitza el nostre tauler en Trello.

Final del sprint: Aquesta fase és on es revisa el treball completat durant el desenvolupament del sprint. Hi ha la possibilitat que s'afegeixin noves tasques en el backlog per a futurs sprints.

Seguint aquestes fases en el projecte, el tauler també té aquestes seccions:

Backlog: És la secció on s'emmagatzemen totes les tasques pendents del projecte, independentment del sprint.

Blocked: És la secció on es guarden aquelles tasques que estan bloquejades pel fet que tenen un problema sense una solució clara temporalment.

Ready: Aquí es guarden les tasques que estan llestes per a fer en l'sprint actual, però que no s'estan treballant en el moment.

In Progress: Aquí es guarden les tasques que s'estan treballant per a completar-se.

Done: Aquí és on es guarden les tasques que s'han finalitzat. Al començament del següent sprint, queden arxivades.

4.2 Desenvolupament

El desenvolupament del projecte utilitzarà diverses metodologies de bones pràctiques per a la programació:

- Test-Driven Development: Aquesta pràctica es dedica a primer desenvolupar els tests de l'aplicació, després implementar el codi i posteriorment refactoritzar (Red-Green-Refactor).

- Domain-Driven Design: Aquesta pràctica es dedica a alinear el codi de l'aplicació amb la lògica de negoci, bàsicament utilitzant un model de domini i llenguatge ubic.

- Arquitectura hexagonal: Juntament amb DDD, es dedica a utilitzar ports i adaptadors per a separar la lògica de negoci de la infraestructura externa.

- Principis SOLID: Són el conjunt de bones pràctiques del disseny orientat a objectes per a mantenir un codi mantenible, flexible i escalable.

5. Definició de requeriments funcionals i tecnològics

5.1. Requeriments funcionals

- Permetre el registre i l'inici de sessió en un compte per accedir a la pàgina web.
- Permetre la introducció de dades d'una sessió de tractament amb una màquina INDIBA mitjançant un formulari web.
- Permetre la introducció de dades de la rehabilitació esportiva del pacient mitjançant un formulari web.
- Permetre la introducció de dades d'informació de PNI del pacient mitjançant una API.
- Validar les dades abans d'emmagatzemar-les a la base de dades de l'aplicació.
- Habilitar la creació de nous pacients per guardar les sessions de tractament.
- Permetre consultar el seguiment complet de la rehabilitació d'un pacient.
- Filtrar i ordenar les sessions segons criteris concrets com la data, la durada o el tipus de tractament.
- Modificar, actualitzar o eliminar sessions en cas que les dades siguin incorrectes.
- Proporcionar una interfície frontend que sigui visualment agradable i senzilla d'utilitzar.
- Generar una auditoria per guardar els registres de transaccions de l'aplicació.
- Crear rols amb diferents nivells de privilegi:
  - ADMIN: Gestió global del sistema i configuració
  - USER: Només ús normal del sistema, generalment només el tindran els fisioterapeutes.
  - AUDITOR: Accés a l'auditoria, compliment de RGPD i traçabilitat.

5.2. Requeriments tecnològics

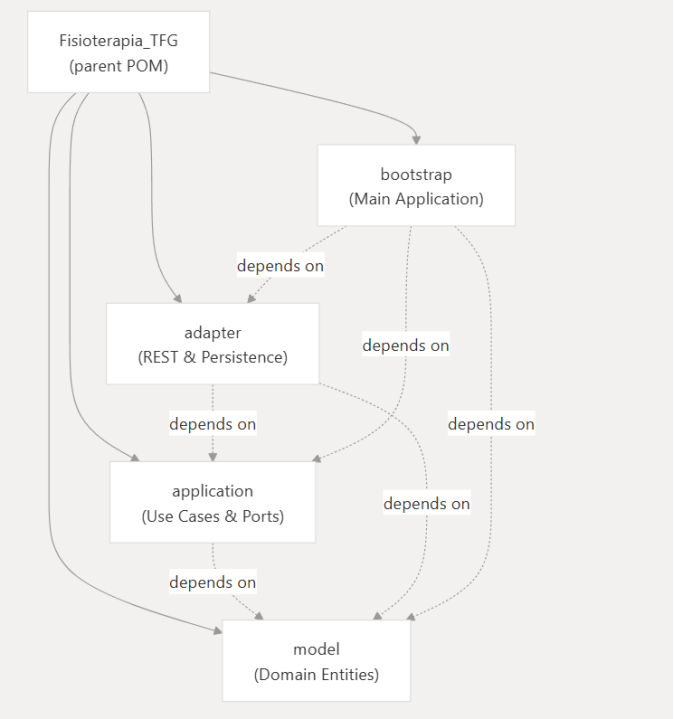
- Utilitzar IntelliJ IDEA com a entorn per el desenvolupament backend de la aplicació
- Utilitzar Visual Studio Code com a entorn per el desenvolupament fronted de la aplicació
- Utilitzar Spring Boot com a framework per el backend de l'aplicació
- Implementar arquitectura hexagonal i DDD per fer l'estructura backend de la aplicació

- Implementar pràctiques de TDD en la aplicació
- Aplicar els principis SOLID en l'aplicació.
- Utilitzar JUnit per fer les proves de l'aplicació
- Utilitzar TestContainers per utilitzar bases de dades no relacionals en testing.
- Utilitzar GitHub com a control de versions
- Utilitzar GitHub Actions per assegurar CI/CD
- Utilitzar Polar AccessLink com a API per obtenir informació de PNI dels pacients.
- Utilitzar HTML5 per fer la estructura frontend de l'aplicació
- Utilitzar CSS3 per fer el disseny del frontend de l'aplicació
- Utilitzar React com a framework per el frontend de l'aplicació

6. Desenvolupament

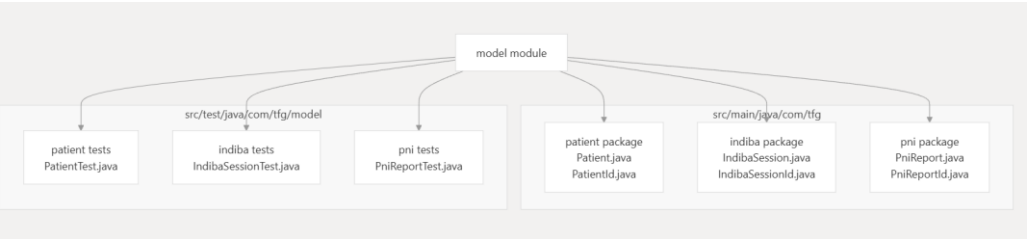
6.1 Inicialització del projecte

Per al desenvolupament d'aquesta aplicació, s'han seguit els passos per a respectar l'Arquitectura Hexagonal segons la guia de Sven Woltman. A més, s'ha utilitzat una plantilla basica de Spring que ve en el IDE d'IntelliJ IDEA i s'ha modificat per a generar els diferents mòduls que farà servir el backend del projecte, afegint les llibreries necessàries a cadascuna. En aquest cas, tenim 4 mòduls: *model*, *bootstrap*, *application* i *adapter*.



6.1.13. Diagrama Arquitectura Hexagonal del projecte. Font: Elaboració pròpia

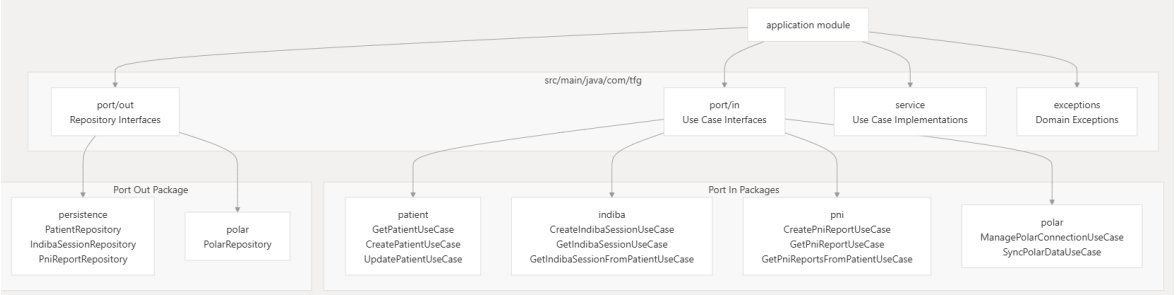
- *model*: on es guarden tots els models de negoci de l'aplicació; no hi ha lògica darrere, excepte per a validar que les dades siguin correctes.



6.1.14. Diagrama mòdul de model. Font: Elaboració pròpia

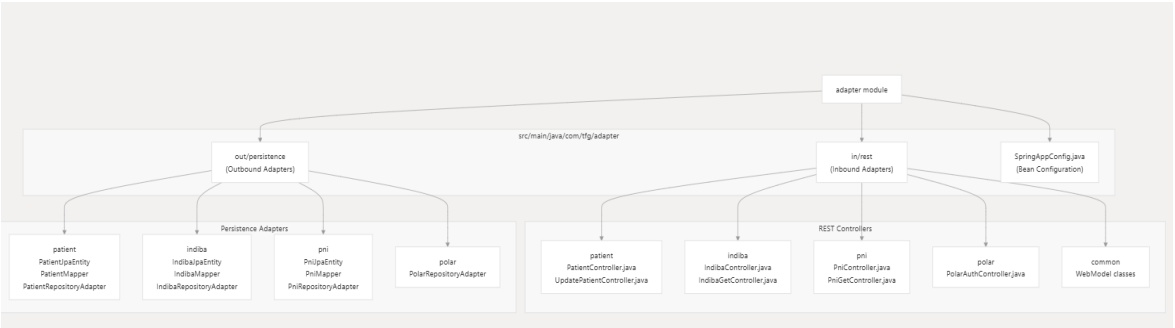
- *bootstrap*: on és instanciant els adaptadors i els serveis de domini de l'aplicació. També inicialitza el servidor web.

- *application*: on es guarda la lògica de l'aplicació que no utilitza cap tecnologia externa; aquí també es guarden els ports i els casos d'ús.



6.1.15. Diagrama de mòdul application. Font: Elaboració pròpia

- *adapter*: on es guarda la lògica que fa servir tecnologia externa; aquí es troben els adaptadors de persistència, REST i Polar.



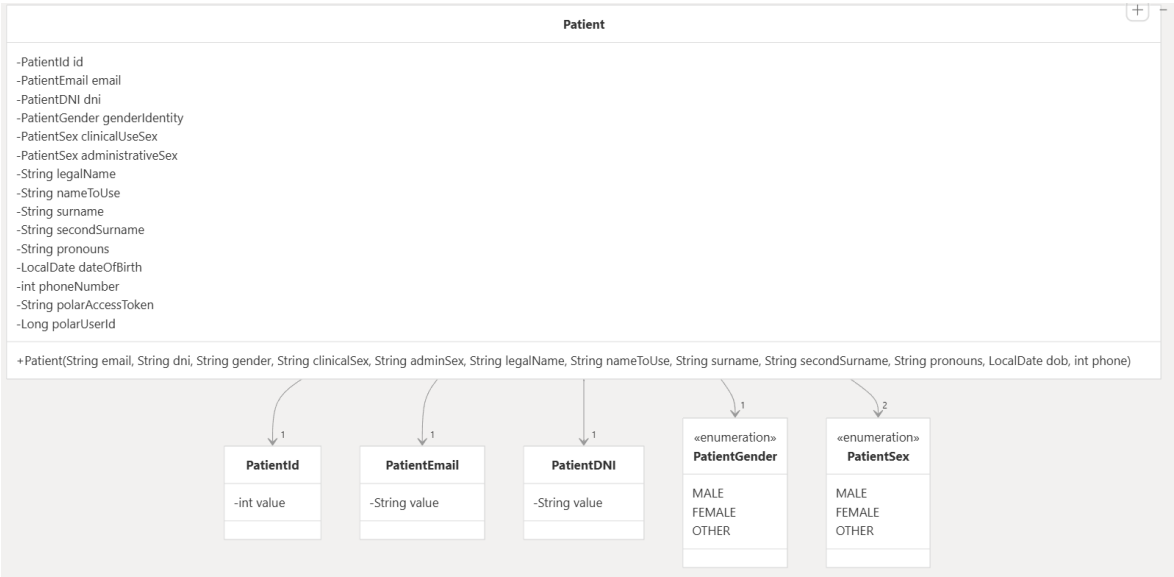
6.1.16. Diagrama de mòdul adapter. Font: Elaboració pròpia

6.2 Dominis de negoci

6.2.1 Pacient

El model de domini del pacient serveix com a component central que representa els individus sotmesos a fisioteràpia. Emmagatzema dades d'identificació, informació de contacte i dades necessàries per a la integració amb sistemes externs.

L'entitat del pacient funciona com el punt de referència principal per a altres components del sistema, incloent-hi Indiba, PNI i Sessions d'entrenament. Aquests components utilitzen l'entitat del pacient per fer un seguiment de les activitats de rehabilitació, les dades fisiològiques i les connexions amb els dispositius wearables.

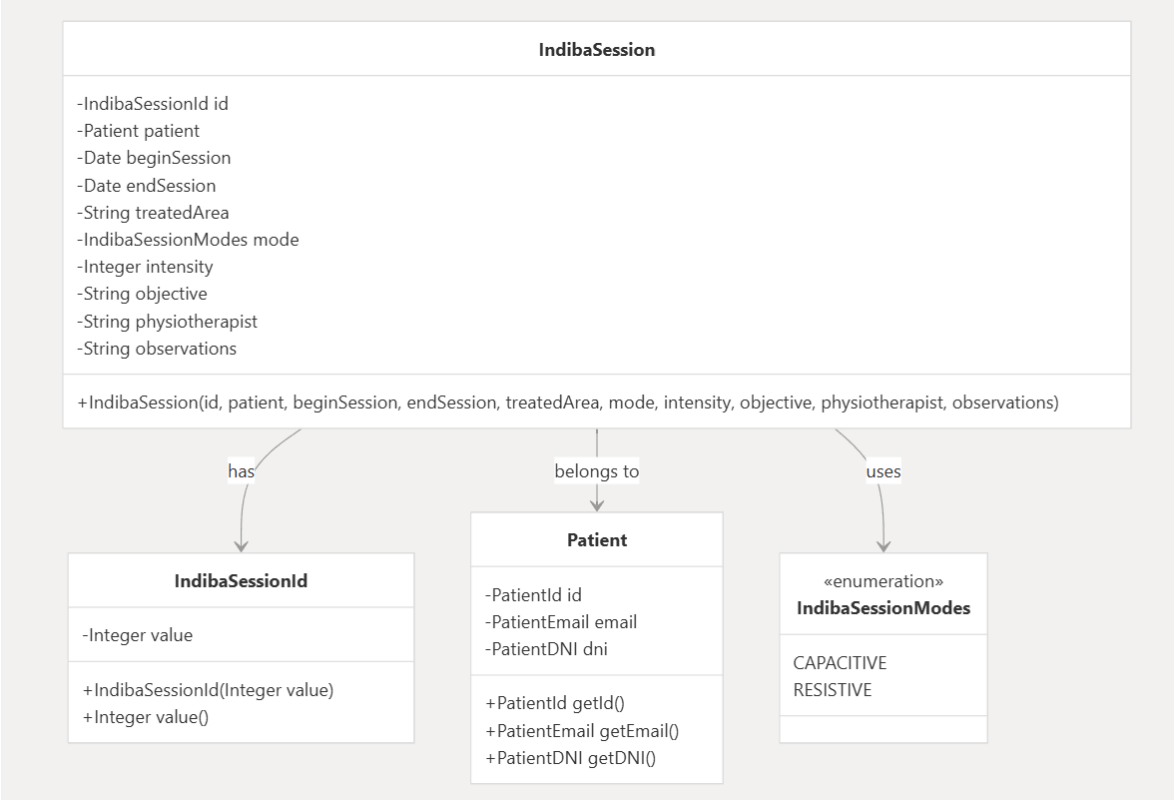


6.2.17. Diagrama d'entitat Patient. Font: Elaboració pròpia

6.2.2 Sesions Indiba

Les sessions Indiba són tractaments individuals en els quals els fisioterapeutes apliquen la radiofreqüència a les àrees dirigides del cos. Es poden ajustar paràmetres com la intensitat del tractament, el mode operatiu i els objectius terapèutics per a cada sessió.

L'entitat *IndibaSession* representa una única instància de radiofreqüència, documentant tant el moment com els paràmetres específics del tractament.

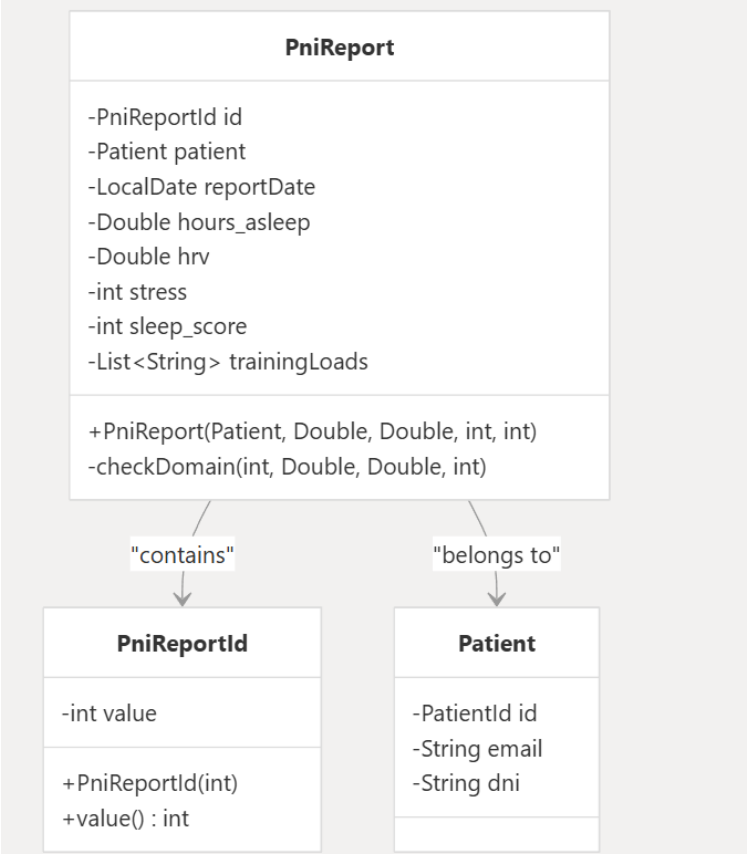


6.2.18. Diagrama d'entitat IndibaSession. Font: Elaboració pròpia

6.2.3 Reports PNI

Els informes del PNI fa un seguiment de les mètriques de salut dels pacients sotmesos a teràpia física a llarg termini. Aquests informes proporcionen dades sobre la durada del son, la variabilitat de la freqüència cardíaca, els nivells d'estrès i la qualitat general del son.

Cada informe PNI s'associa a un pacient individual i es pot generar diàriament utilitzant dades recollides de dispositius portàtils polars.

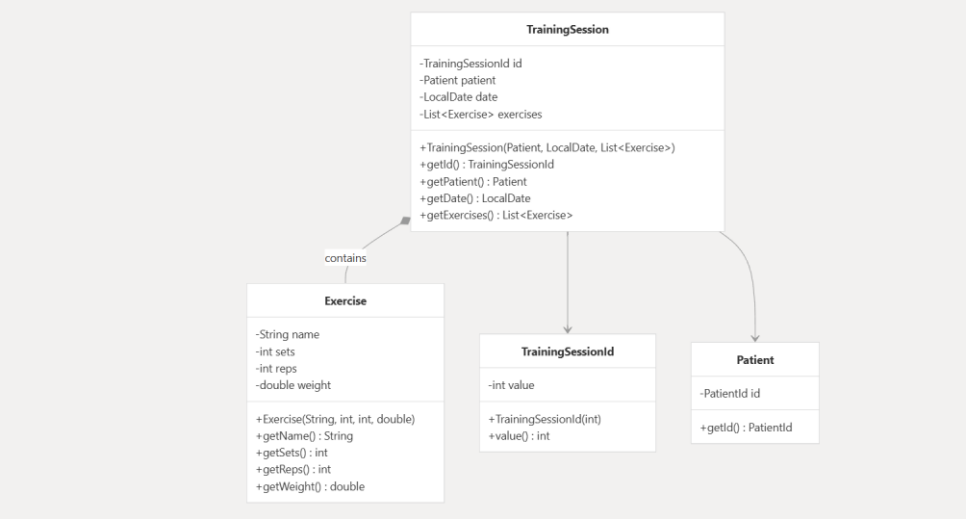


6.2.19. Diagrama d'entitat PniReport. Font: Elaboració pròpia

6.2.4 Sesions de entrenament

El domini Training Sessions supervisa les sessions de rehabilitació esportiva i supervisa els exercicis dels pacients en fisioteràpia. Documenta programes estructurats d'exercici prescrits per fisioterapeutes, especificant detalls per a cada exercici, incloent-hi el nom, conjunts, repeticions i pes.

L'àrea Training Sessions comprèn dos components principals: TrainingSession i Exercise. Aquests components s'organitzen d'acord amb principis de disseny específics i es mantenen per separat de la configuració tècnica. A més, un registre ExerciseSet proporciona més detalls sobre aquesta àrea.



6.2.20 Diagrama d'entitat TrainingSession. Font: Elaboració pròpia

6.3 Casos d'ús

6.3.1 Gestió del Cicle de vida de les entitats

A diferència dels sistemes tradicionals que només permeten crear, llegir, actualitzar o esborrar dades, aquest projecte es fonamenta en la gestió de tasques i en la preservació íntegra de les dades antigues. En l'àmbit de la fisioteràpia, tant clínica com esportiva, els esdeveniments passats, com ara les sessions d'entrenament i les dades de càrrega dels pacients, constitueixen registres històrics que han de ser conservats sense modificacions.

Per aquest motiu, el sistema prioritza l'afegiment i la consulta de dades, i evita de manera explícita les actualitzacions generals per tal de preservar l'historial del pacient sense alteracions accidentals.

Operacions de creació i registre (Create)

Té com a propòsit registrar noves entitats en el sistema, com l'alta d'un nou pacient o el registre d'una sessió d'entrenament.

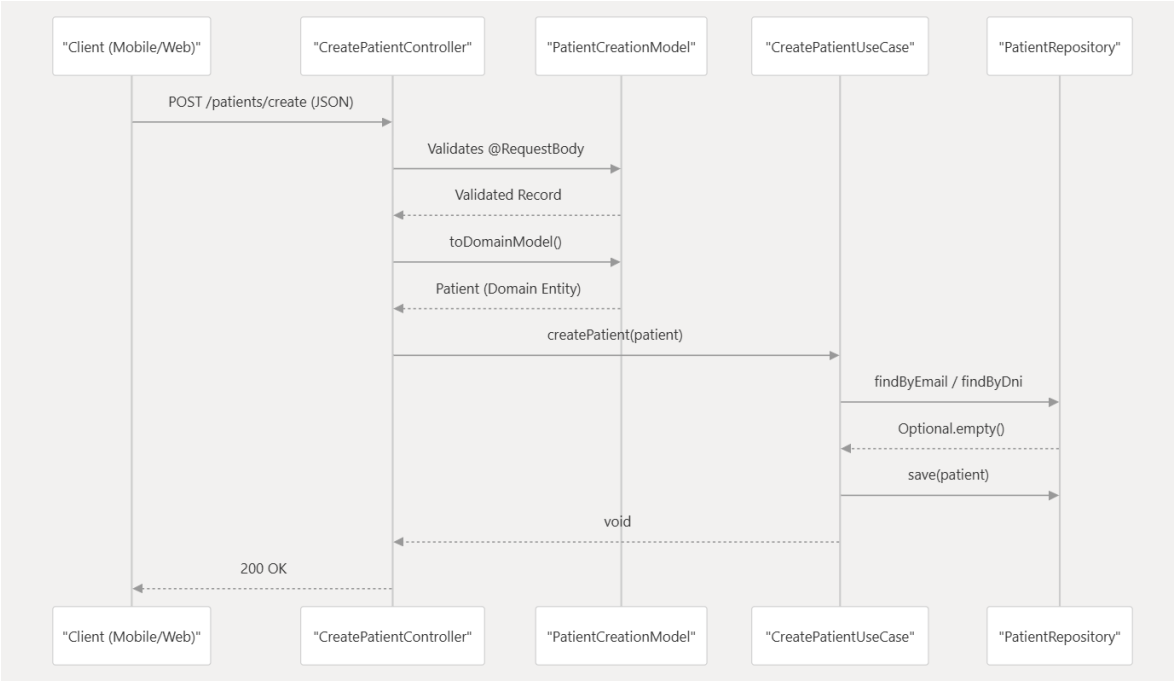
Flux:

1. L'Adaptador d'Entrada (Controlador REST) rep la petició HTTP amb un Data Transfer Object (DTO) i el mapeja al model de domini.

2. El Cas d'Ús (per exemple, CreateTrainingSessionUseCase) orquestra l'operació, validant les invariants i regles de negoci estrictes, com l'existència prèvia del pacient referit en el sistema.

3. Després de validar l'estat, el Cas d'Ús delega la persistència al Port de Sortida (Repositori), que utilitza el seu Adaptador JPA per a interactuar amb la base de dades subjacent.

Com a resposta, es retorna l'identificador del recurs generat juntament amb el codi d'estat HTTP 200 (OK).



6.3.21. Flux de treball per CreatePatient. Font: Elaboració pròpia

Operacions de Consulta i Anàlisi (Read)

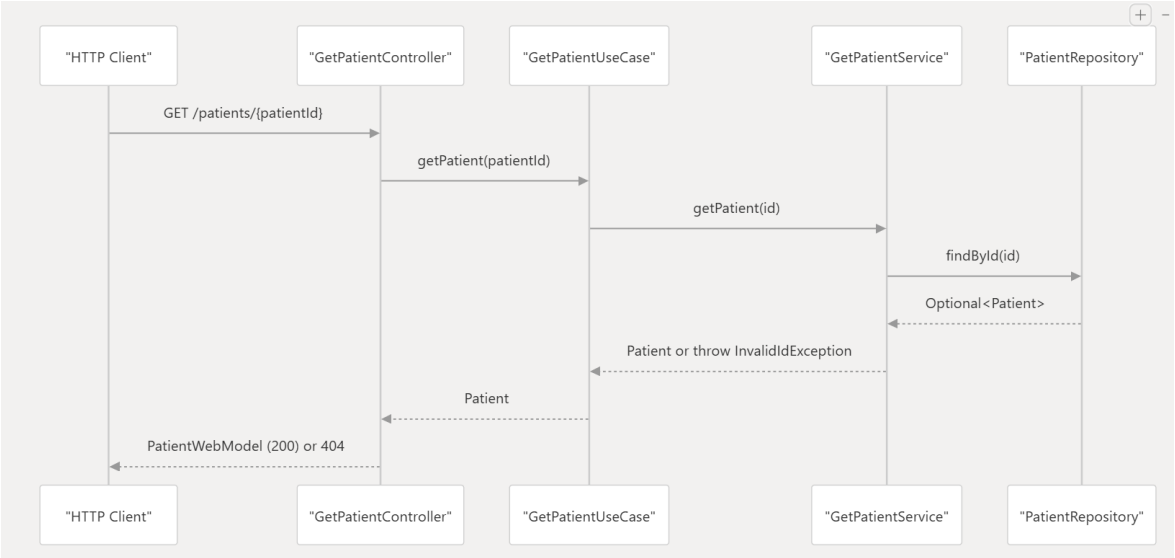
El propòsit consisteix a recuperar informació del domini i calcular mètriques de progressió, tot aïllant la complexitat associada a les consultes del client.

Flux:

1. El controlador invoca casos d'ús de lectura especialitzats, com GetPatientUseCase.

2. El cas d'ús consulta el repositori mitjançant identificadors únics. Si l'entitat no existeix, el sistema llança una excepció InvalidIdException. Un manegador global d'excepcions (GlobalExceptionHandler) converteix aquesta excepció en un error HTTP 404.

3. Les dades del domini, o les seves projeccions com a sumatoris del volum de càrrega, es transformen en DTOs de resposta. Aquest procés garanteix que el client rebi únicament la informació estrictament necessària.



6.3.22. Flux de treball de GetPatient. Font: Elaboració pròpia

Consulta d'Historial i Llistats Cronològics

El propòsit d'aquest sistema és recuperar i mostrar l'historial d'intervencions d'un pacient, com a sessions d'entrenament, informes de psiconeuroimmunologia (PNI) o sessions de Indiba.

Flux:

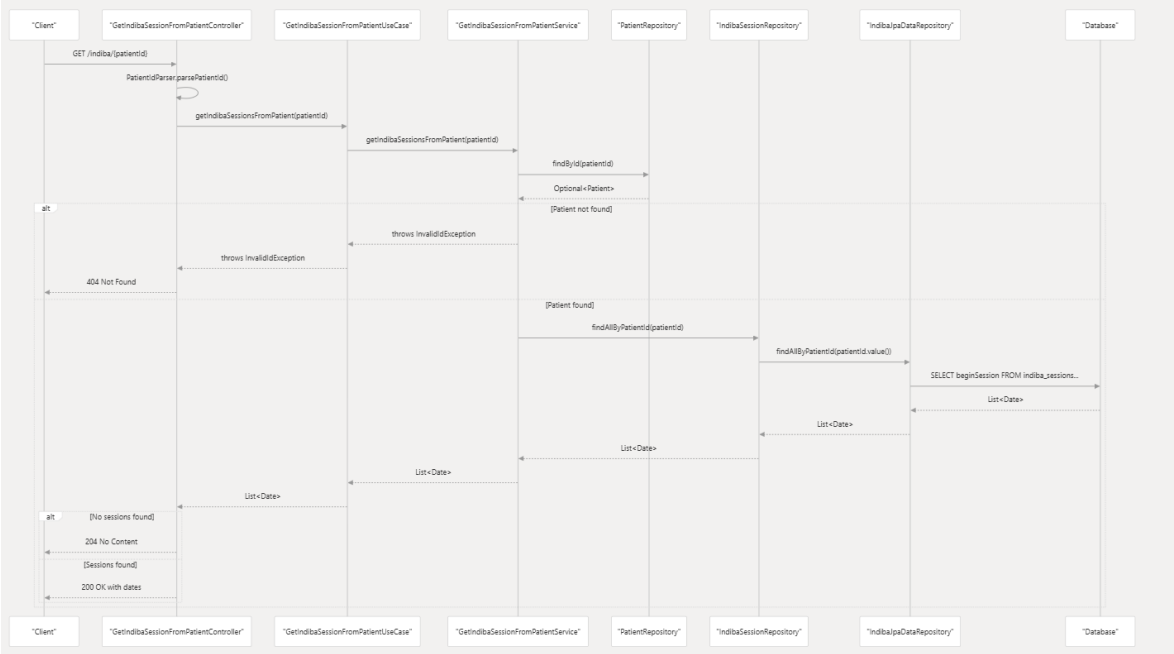
1. L'Adaptador d'Entrada (Controlador REST) rep la sol·licitud HTTP GET que inclou l'identificador únic del pacient.

2. El Cas d'Ús específic (per exemple, GetPatientTrainingSessionsUseCase o GetPatientPniReportsUseCase) determina l'operació. Primer, revisa si el pacient

està en el sistema; si no està, el procés es deté i es mostra un error (per exemple, InvalidIdException).

3. Se sol·licita la informació al Port de Sortida (Repositori). L'Adaptador JPA realitza cerques ràpides i deixa que el motor de la base de dades s'encarregui de guardar i recuperar les dades.

4. Les entitats recuperades es transformen en versions resumides o en DTOs de resum. Aquest procés de conversió omet detalls específics, com el nombre exacte de sèries i repeticions o el text complet d'un informe clínic, i només inclou la informació més important per a les llistes.



6.3.23. Flux de treball de GetIndibaFromPatient. Font: Elaboració pròpia

6.3.2 Polar API

El sistema d'integració polar recull automàticament dades de salut del pacient dels dispositius portàtils polars. Els pacients connecten els seus dispositius una vegada per donar permís, cosa que permet al sistema actualitzar automàticament la informació diària sobre la qualitat del son, els canvis en la freqüència cardíaca i els nivells d'estrès. Les dades recollides s'organitzen en informes PNI per ajudar a fer un seguiment de la recuperació en el temps.

Creació de pacient al servidor de Polar API

El propòsit és permetre que un pacient connecti el seu Polar wearable en dos passos: primer, inicien el procés a /api/auth/polar/authorize, que els redirigeix a la pàgina de permís de Polar. Després que estiguin d'acord, Polar els redirigeix a /api/auth/polar/callback, on el sistema intercanvia el codi per als testimonis d'accés i els emmagatzema en el registre del pacient.

Flux:

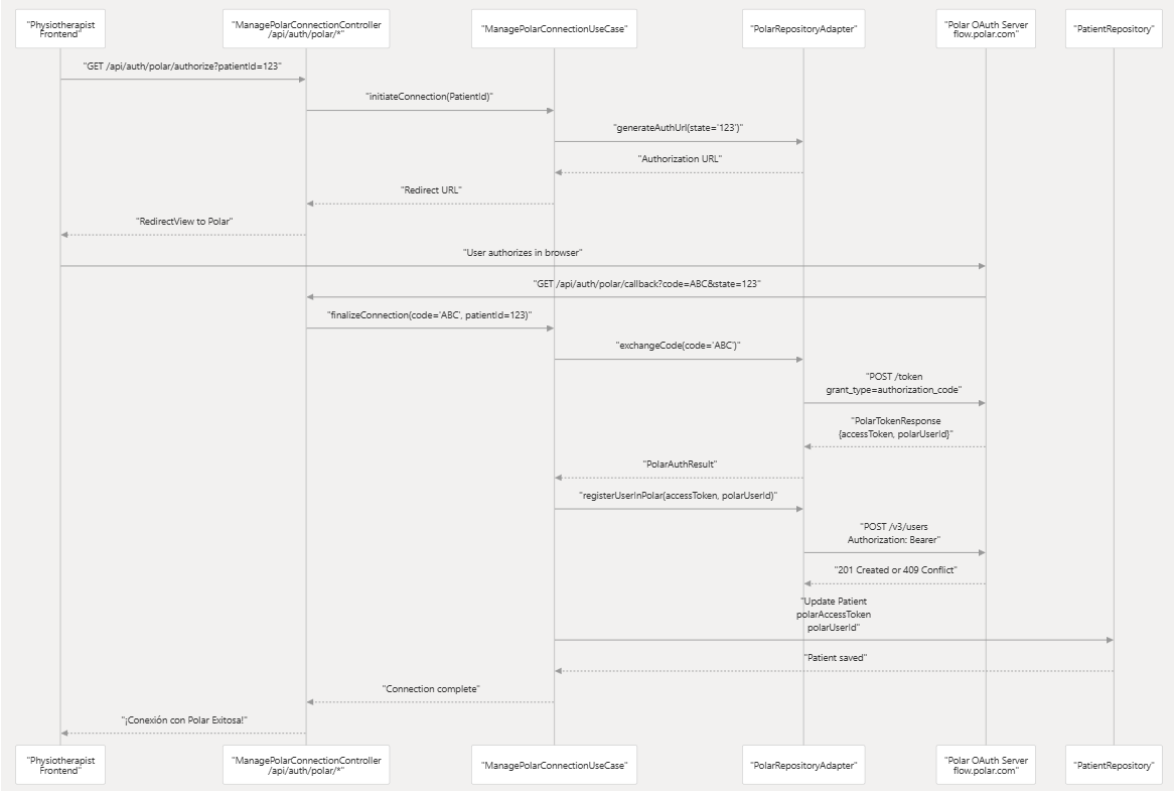
1. El controlador d'entrada (ManagePolarConnectionController) processa la primera sol·licitud de connexió usant l'identificador únic del pacient (patientId).

2. Després de comprovar que el pacient existeix, el sistema sol·licita a l'adaptador de sortida (PolarRepositoryAdapter) que creï la direcció d'autorització. Per a mantenir la seguretat i rastrejar la sessió, l'identificador del pacient s'inclou en el paràmetre state, juntament amb els permisos de lectura necessaris (scope="*accesslink.read"). El sistema després redirigeix a l'usuari a la pàgina de Polar mitjançant un codi HTTP 302 perquè doni el seu consentiment.

3. Després de l'aprovació, el sistema rep la resposta (callback) del servidor Polar. El controlador pren el codi temporal d'autorització (code) i obté l'identificador del pacient en llegir el paràmetre state.

4. El sistema realitza una sol·licitud directa a la API de Polar per a canviar el codi temporal per un token d'accés permanent (polarAccessToken) i per l'identificador de l'usuari (polarUserId).

5. Es busca la informació del pacient en el sistema, s'actualitza amb les noves credencials i es guarda en el repositori JPA.



6.3.24. Flux de treball ManagePolarConnection. Font: Elaboració pròpia

Recollida de reports PNI automàtic

L'aplicació crea automàticament informes PNI mitjançant la recopilació de dades diàries de salut dels dispositius polars connectats. Utilitza un servei per obtenir informació de son i recuperació del sistema de Polar cada dia a les 6 AM, després fa i guarda informes PNI a la base de dades.

Flux:

1. A diferència de les operacions web, aquest procés comença únicament mitjançant un programa anomenat PolarDailyScheduler, que es configura per a executar-se cada dia a les #6:00 AM.

2. El planificador crida a un procés anomenat SyncPolarDataUseCase, que busca a tots els pacients que ja van connectar el seu compte i tenen un permís d'accés vàlid.

3. Per a cada pacient, el procés utilitza un component anomenat PolarRepositoryAdapter per a sol·licitar dades a Polar. El sistema només sol·licita les mètriques de somni i de recuperació del dia anterior.

4. Amb les dades obtingudes, sempre que hi hagi informació vàlida registrada pel dispositiu, el sistema genera un informe anomenat PniReport. Durant la seva creació, es revisen les dades clíniques, com les hores de somni o la puntuació de recàrrega, per a assegurar-se que siguin correctes abans d'incorporar-los al sistema.

5. L'adaptador JPA presa l'informe, converteix les seves dades i relacions al format requerit i els guarda en la taula pni_reports de la base de dades.

6.3.25. Flux de treball PolarDailyScheduler. Font: Elaboració pròpia

6.3.3 Estadístiques i metriques de pacients

Ràtio de Transició Mensual

El seu propòsit és mesurar, amb xifres, com avança el pacient des d'una etapa de tractament passiu en la clínica (sessions de radiofreqüència amb INDIBA) fins a una etapa de rehabilitació activa (sessions d'entrenament). Aquesta mesura junta les dades de cada mes d'un any, la qual cosa ajuda el professional a veure amb claredat el progrés cap a l'alta clínica.

Flux:

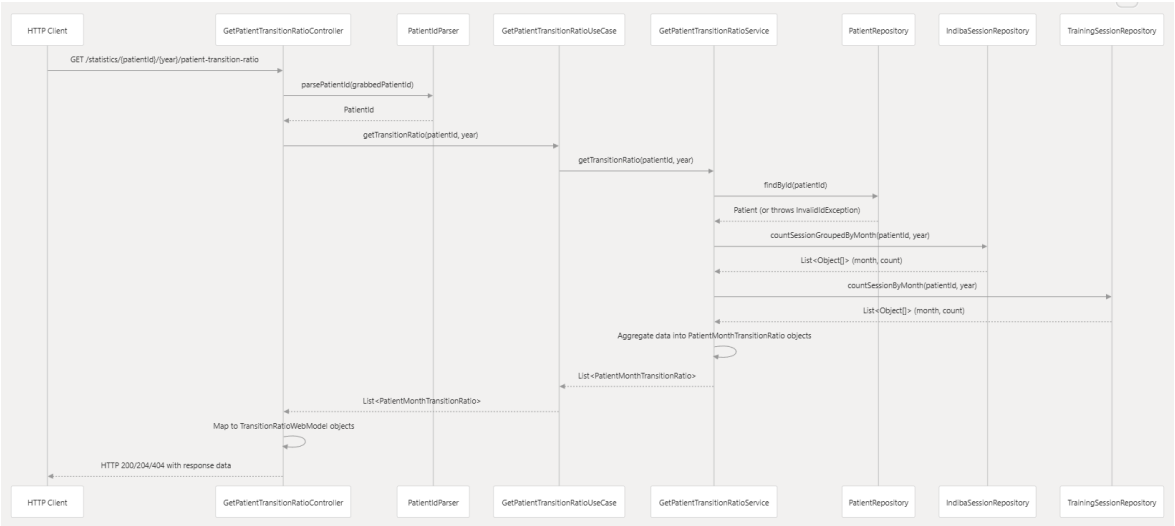
1. L'Adaptador d'Entrada (GetPatientTransitionRatioController) rep una sol·licitud HTTP GET que inclou l'identificador del pacient (patientId) i l'any a consultar (year). Un component transversal (PatientIdParser) verifica que el format de l'identificador sigui un UUID vàlid.

2. El Controlador delega l'execució al Cas d'Ús corresponent (GetPatientTransitionRatioUseCase).

3. Perquè el procés sigui eficient, el Cas d'Ús no carrega totes les sessions en memòria, sinó que utilitza els Ports de Sortida (com IndibaSessionRepository) per a realitzar consultes més ràpides que agrupen per mesos i compten les sessions directament en la base de dades.

4. Amb les dades reunides, es crea el model de domini PatientMonthTransitionRatio. En aquesta entitat principal s'aplica la regla matemàtica: es calcula la proporció de sessions d'entrenament respecte del total de sessions del mes. El model està preparat per a evitar errors, retornant una ràtio de 0.0 si el total de sessions és zero, evitant així una fallada en dividir entre zero.

5. Finalment, el resultat del domini es converteix en un objecte de transferència (TransitionRatioWebModel) per a enviar-lo de manera segura al client.



6.3.26. Flux de treball GetPatientTransitionRatio. Font: Elaboració pròpia

Progressió de Càrrega de Treball

El seu propòsit és seguir i mesurar com canvien, amb el temps, la quantitat o l'esforç que un pacient realitza durant un exercici específic. Aquesta mesura és clau per a comprovar si hi ha augment de massa muscular o millora de la força al final de la rehabilitació.

Flux:

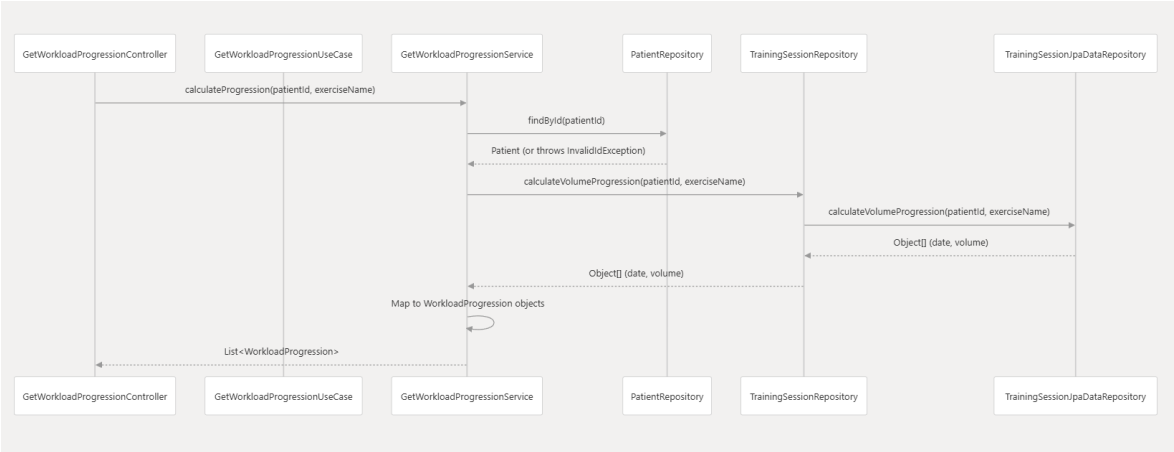
1. L'Adaptador d'Entrada (GetWorkloadProgressionController) rep la sol·licitud HTTP GET que inclou l'identificador del pacient i el nom exacte de l'exercici a revisar, com a "Esquat". L'identificador es revisa acuradament per a assegurar-se que tingui el format UUID correcte.

2. El Cas d'Ús (GetWorkloadProgressionUseCase) gestiona el flux de negoci.

3. El Repositori lliura l'historial de sèries (ExerciseSets) d'aquest pacient i exercici, ordenades per la data de la sessió, mitjançant una consulta.

4. La informació recuperada es passa al registre de domini WorkloadProgression. En aquest component s'utilitza la lògica esportiva per a calcular la càrrega de treball total, que normalment s'obté multiplicant les sèries, les repeticions i el pes aixecat de cada data registrada.

5. Les entitats de progressió calculades es transformen en representacions visuals (DTOs) que permeten a la interfície gràfica mostrar els gràfics d'evolució.



6.3.27. Flux de treball GetWorkflowProgression. Font: Elaboració pròpia

8. Ampliacions

Integració del sistema amb el programari oficial dels dispositius INDIBA que permet la recollida automàtica de dades de tractament. Aquesta automatització eliminaria la necessitat d'introducció manual de dades per part del fisioterapeuta, fet que garanteix una precisió més elevada en el registre d'informació rellevant com la intensitat, el mode i el temps, i facilita que el professional pugui dedicar més temps a l'atenció del pacient.

Integració de plataformes d'agregació com 'Open Wearables' per augmentar la compatibilitat del sistema. Aquesta acció suposaria substituir la connexió directa actual (API de Polar) per un sistema obert capaç de recollir dades biomètriques de qualsevol marca de dispositiu comercial. Aquesta millora permetria que el sistema fos independent del fabricant i facilitar l'accés als pacients que utilitzen dispositius de diferents rangs de preu i qualitat.

8. Bibliografia

[1] A. Cockburn.: Hexagonal Architecture.
[en línia] [consulta: 13 de novembre de 2025].
Disponible a: https://alistair.cockburn.us/hexagonal-architecture

[2] HappyCoders.eu: Hexagonal Architecture – Software Craftsmanship.
[en línia] [consulta: 13 de novembre de 2025].
Disponible a: https://www.happycoders.eu/software-craftsmanship/hexagonal-architecture/

[3] Chuidiang: TDD – Test Driven Development.
[en línia] [consulta: 13 de novembre de 2025].
Disponible a: https://old.chuidiang.org/java/herramientas/test-automaticos/tdd-test-driven-development.php

[4] J.Salomón: *JonathanM2ndoza/Hexagonal-Architecture-DDD.* [en línia] [consulta: 20 d'Octubre de 2025]. Disponible a: https://github.com/JonathanM2ndoza/Hexagonal-Architecture-DDD

[5] DomingoAlvarez99: Hexagonal Architecture Example. GitHub.
[en línia] [consulta: 20 d'octubre de 2025].
Disponible a: https://github.com/DomingoAlvarez99/hexagonal-architecture-example

[6] Alican. A: Modular Architecture Hexagonal Demo Project. GitHub.
[en línia] [consulta: 20 d'octubre de 2025].
Disponible a: https://github.com/AlicanAkkus/Modular-Architecture-Hexagonal-Demo-Project

[7] Atlassian: What is version control.
[en línia] [consulta: 4 de novembre de 2025].
Disponible a: https://www.atlassian.com/es/git/tutorials/what-is-version-control

[8] Universidad Nacional de La Plata: Trabajo académico sobre desarrollo de software (repositorio SEDICI).
[en línia] [consulta: 4 de novembre de 2025].

Disponible a: https://sedici.unlp.edu.ar/handle/10915/120476

[9] Spring: Spring Boot Project.
[en línia] [consulta: 4 de novembre de 2025].
Disponible a: https://spring.io/projects/spring-boot

[10] Apache Software Foundation: What is Maven.
[en línia] [consulta: 4 de novembre de 2025].
Disponible a: https://maven.apache.org/what-is-maven.html

[11] Spring: Spring Data Project.
[en línia] [consulta: 4 de novembre de 2025].
Disponible a: https://spring.io/projects/spring-data

[12] Spring: Spring Data JPA.
[en línia] [consulta: 4 de novembre de 2025].
Disponible a: https://spring.io/projects/spring-data-jpa

[13] Spring: Spring Data MongoDB.
[en línia] [consulta: 4 de novembre de 2025].
Disponible a: https://spring.io/projects/spring-data-mongodb

[14] SmartBear: Swagger API Platform.
[en línia] [consulta: 4 de novembre de 2025].
Disponible a: https://swagger.io/

[15] GitHub: Collaboration Features.
[en línia] [consulta: 4 de novembre de 2025].
Disponible a: https://github.com/features#features-collaboration

[16] GitLab: GitLab DevOps Platform.
[en línia] [consulta: 4 de novembre de 2025].
Disponible a: https://about.gitlab.com

[17] H2 Database Engine: Database features.
[en línia] [consulta: 4 de novembre de 2025].

Disponible a: https://www.h2database.com/html/features.html

[18] P. Preston.: *HTML 5*. Babelcube Inc., 2015.
[en línia] [consulta: 4 de novembre de 2025].
Disponible a: Google Books

[19] Mar de Asía: *Manual de CSS3*.
[en línia] [consulta: 4 de novembre de 2025].
Disponible a: https://www.mardeasa.es/descargas/recursos-paginas-web/css/manuales/manual-css3-nov2014.pdf

[20] Meta Open Source: *React Documentation*.
[en línia] [consulta: 4 de novembre de 2025].
Disponible a: https://es.react.dev/

[21] C. Gackenheimer: *Introduction to React*. Apress Media, New York, 2015, ISBN: 978-1-4842-1246-2

[22] Google*: AngularJS Framework*.
[en línia] [consulta: 4 de novembre de 2025].
Disponible a: https://angularjs.org/

[23] N. Jain, P. Mangal, D. Mehta: AngularJS: A modern MVC framework in JavaScript.
[en línia] [consulta: 4 de novembre de 2025].
Disponible a: https://www.academia.edu/80043820/AngularJS_A_Modern_MVC_Framework_in_JavaScript

[24] JUnit Team: JUnit Framework.
[en línia] [consulta: 4 de novembre de 2025].
Disponible a: https://junit.org/

[25] JUnit Team: JUnit User Guide.
[en línia] [consulta: 4 de novembre de 2025].
Disponible a: https://docs.junit.org/current/user-guide/

[26] Testcontainers: Testcontainers for Java.
[en línia] [consulta: 4 de novembre de 2025].
Disponible a: https://testcontainers.com/

[27] Testcontainers: Getting Started with Testcontainers for Java.
[en línia] [consulta: 4 de novembre de 2025].
Disponible a: https://testcontainers.com/guides/getting-started-with-testcontainers-for-java/

[28] GitHub: GitHub Actions.
[en línia] [consulta: 5 de novembre de 2025].
Disponible a: https://github.com/features/actions

[29] GitHub: Understanding GitHub Actions.
[en línia] [consulta: 5 de novembre de 2025].
Disponible a: https://docs.github.com/es/actions/get-started/understand-github-actions

[30] GitLab: Auto DevOps Documentation.
[en línia] [consulta: 5 de novembre de 2025].
Disponible a: https://docs.gitlab.com/topics/autodevops/

[31] JetBrains: IntelliJ IDEA.
[en línia] [consulta: 5 de novembre de 2025].
Disponible a: https://www.jetbrains.com/es-es/idea/

[32] JetBrains: IntelliJ IDEA Features.
[en línia] [consulta: 5 de novembre de 2025].
Disponible a: https://www.jetbrains.com/es-es/idea/features/

[33] Microsoft: Visual Studio Code.
[en línia] [consulta: 5 de novembre de 2025].
Disponible a: https://code.visualstudio.com/

[34] Atlassian: *Trello.*
[en línia] [consulta: 5 de novembre de 2025].
Disponible a: https://trello.com/es

[35] Atlassian: *GitHub and Trello Integration.*
[en línia] [consulta: 5 de novembre de 2025].
Disponible a: https://www.atlassian.com/blog/trello/github-and-trello-integrate-your-commits

[37] YH. Lim, SH. Wong: *Effects of isometric, eccentric, or heavy slow resistance exercises on pain and function in individuals with patellar tendinopathy: A systematic review* [en línia] [consulta: 20 de novembre de 2025]. Disponible a: https://pubmed.ncbi.nlm.nih.gov/29972281/

[38] R. Phillips, C. Hilton, LF. Sousa Filho, M. Farlie, D. Morrissey, P. Malliaras: *Behaviour change and rehabilitation adherence in adults with tendinopathy: a scoping review* [en línia] [consulta: 20 de novembre de 2025]. Disponible a: https://pubmed.ncbi.nlm.nih.gov/38420953/

[39] M. Cuevas-Cervera, D. Aguilar-Núñez, M. Aguilar-García, MC. García-Ríos, A. González-Muñoz, S. Navarro-Ledesma: Patellar tendon elasticity and temperature following after a 448 kilohertz radiofrequency intervention on active healthy subjects: an open controlled clinical trial [en línia] [consulta: 23 de novembre de 2025]. Disponible a: https://www.mdpi.com/2075-4418/13/18/2976

[40] B. Kumaran, T. Watson: *Treatment using 448kHz capacitive resistive monopolar radiofrequency improves pain and function in patients with osteoarthritis of the knee joint: a randomised controlled trial* [en línia] [consulta: 23 de novembre de 2025]. Disponible a: https://pubmed.ncbi.nlm.nih.gov/30269963/

[41] F. Abat, J. Torras, A. Garcia, E. Jordán, M. Roby, R. Yáñez, C. De la Fuente: Pain Decrement Using Radiofrequency Therapy After Knee Platelet-Rich Plasma Injections Within First 72 h in Active Populations with Patellar Chondropathy [en línia] [consulta: 23 de novembre de 2025]. Disponible a: https://www.mdpi.com/2077-0383/14/2/544

[42] I. I-Zouikr, KD. Beck: Lifetime modulation of the pain system via neuroimmune and neuroendocrine mechanisms [en línia] [consulta: 24 de novembre de 2025]. Disponible a: https://pmc.ncbi.nlm.nih.gov/articles/PMC5347117/

[43] M. Ayaz, MF. Pasha, MY. Alzahrani, R. Budiarto, D. Stiawan: The Fast Health Interoperability Resources (FHIR) Standard: Systematic Literature Review of Implementations, Applications, Challenges and Opportunities [en línia] [consulta: 25 de novembre de 2025]. Disponible a: https://pmc.ncbi.nlm.nih.gov/articles/PMC8367140/

[44] M. Schwarz *et al*: Digital mental health and hidden support: a qualitative analysis of non-suicidal self. [en línia] [consulta: 26 de novembre de 2025]. Disponible a: https://www.frontiersin.org/journals/digital-health/articles/10.3389/fdgth.2025.1636775/full

[45] Open Wearables [en línia] [consulta: 30 de novembre de 2025]. Disponible a: https://www.openwearables.io/

[46] Polar AccessLink API v3 [en línia] [consulta: 11 de Març de 2026]. Disponible a: https://www.polar.com/accesslink-api/?java#polar-accesslink-api